CyberAI Security Assistant

System Design Document (Security Brain - Version 7)

Author: Muhammad Shamshair

---

1. Project Overview

What is CyberAI?

CyberAI is a **fully local AI-powered Offensive Security Assistant** developed for cybersecurity professionals, penetration testers, application security engineers, and security researchers.

Unlike cloud AI assistants, CyberAI is designed to execute completely inside a private Kali Linux environment, ensuring that:

- Source code never leaves the local machine.
- Internal assessments remain confidential.
- Sensitive applications are analyzed offline.
- AI reasoning stays under the user's control.

The objective is not to replace the penetration tester, but to function as a **Senior Security Consultant** that assists during every stage of a security assessment.

CyberAI Design Philosophy

- Privacy by Design
- Local AI First
- Modular Components
- Explainable Security Decisions
- Human-in-the-Loop
- Evidence-Based Reporting
- Least-Privilege Tool Execution
- Offline Operation

2. Primary Purpose

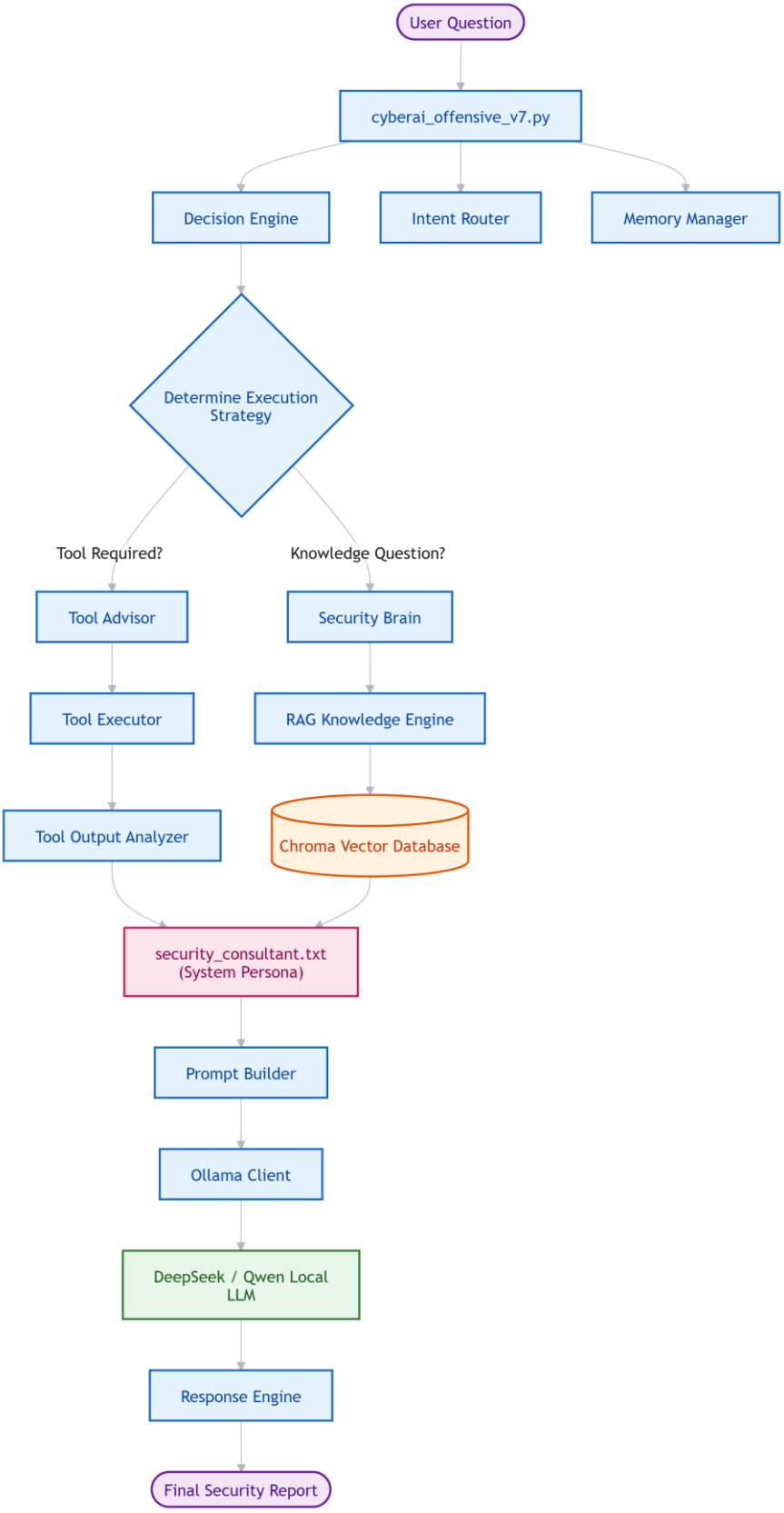
CyberAI was created to become a personal security consultant capable of assisting with:

- Web Application Security
- Mobile Application Security
- API Security
- Network Security
- Cloud Security
- DevSecOps
- Malware Analysis

- Reverse Engineering
- Threat Modeling
- Red Teaming
- OSCP-style methodology
- OSWE methodology
- Secure Coding
- Vulnerability Research
- CTF learning
- Security report generation

The long-term vision is to evolve CyberAI into a complete local cybersecurity operating environment with advanced reasoning and automation.

3. High-Level Architecture



4. Core Components

4.1 Main Controller

File: cyberai_offensive_v7.py

Responsibility

This is the orchestrator. It never performs security analysis itself. Instead it coordinates all modules.

Responsibilities include:

- Accept user questions
- Validate input e.g. ;, &&, ()
- Call Decision Engine
- Call Security Brain
- Execute security tools
- Search RAG
- Search Memory
- Build AI prompt
- Call Ollama
- Validate AI output
- Display final report

4.2 Decision Engine

Module: decision_engine.py

Determines whether to execute tools, RAG, memory or LLM.

Purpose

Acts as the "brain of execution."

It decides:

- Should RAG run?
- Should DeepSeek run?
- Should a security tool execute?
- Is the question too small for AI?
- Is this only tool analysis?

Example:

```
scan localhost with nmap
```

Decision:

```
Run Tool
Skip DeepSeek
Analyze locally
```

This dramatically reduces response time.

4.3 Intent Router

Module: intent_router.py

Purpose:

Identify what the user is asking.

Example categories:

```
Security Knowledge
Tool Request
Security Analysis
Learning
Code Review
Report Generation
Threat Modeling
```

This classification drives the Decision Engine.

4.4 Security Brain

Module: security_brain.py

This is one of the most important modules.

Generic LLMs provide conversational answers but often lack a structured penetration-testing methodology. The Security Brain injects offensive-security reasoning before LLM inference, ensuring that responses consistently consider attack surface, attacker objectives, validation strategy, evidence requirements, business impact, and remediation.

It provides structured context such as:

- Security domain
- Attack surface
- Attacker objectives
- Attack techniques
- Testing methodology
- Recommended tools
- Evidence requirements
- Risk areas
- Business impact
- Remediation focus

It also includes a "pentester mindset" that encourages:

- Reconnaissance strategy
- Attack hypotheses
- Validation strategy
- Evidence collection
- False-positive checks

This ensures responses follow a practical offensive-security methodology rather than generic AI explanations.

4.5 Tool Advisor

Module: tool_advisor.py

Purpose:

Recommend the appropriate security tool.

Examples:
Question:
```
Run Nmap
```
↓
Output:
```
Tool: Nmap
```

Question:
```
SSL Pinning
```
↓
Output:
```
Frida
Objection
MobSF
```

Question:
```
Web Assessment
```
↓
Output:
```
Burp Suite
```

4.6 Tool Executor
Module: tool_executor.py

Purpose:
Safely execute approved tools.

Current whitelist includes tools such as:
- Nmap
- Nikto
- WhatWeb
- Whois

Future additions may include other security utilities after careful review.
Safety features:
- Whitelisted tools only
- Timeout control
- Output capture
- Error handling

4.7 Tool Output Analyzer
Module: tool_output_analyzer.py

Purpose:
Convert raw tool output into structured security findings.

Example:
Raw Nmap
↓

```
Port 80
HTTP
OpenWebUI
Risk
Recommendation
```

This avoids sending large raw outputs to the LLM when a concise local analysis is sufficient.

4.8 Knowledge Base + RAG

Module: rag_engine.py

Purpose:
Provide factual cybersecurity knowledge retrieved from a local vector database before AI reasoning.

Current knowledge domains include:

- OSCP
- OSWE
- OWASP
- OWASP WSTG
- OWASP MSTG
- API Security
- Network Security
- Red Team
- CTF
- Android Security
- iOS Security
- Cloud Security
- Kubernetes
- Secure Coding
- SAST
- VAPT
- Threat Modeling
- MITRE ATT&CK
- Malware Analysis
- Reverse Engineering
- CVEs
- CWEs
- Exploit Development
- Bug Bounty
- Digital Forensics
- Security Tools

The knowledge is embedded locally and stored in ChromaDB, allowing semantic retrieval without internet access.

4.9 Memory Manager
Module: memory_manager.py

Purpose:
Store and retrieve previous interactions.

Functions:

- Save questions
- Save answers
- Search previous discussions
- Reuse historical context

Unlike the RAG knowledge base, this contains **user-specific conversation history** rather than general cybersecurity references.

4.10 security_consultant.txt
Defines CyberAI's permanent security consultant persona, behavior, reporting style, methodology and response format.
It is loaded as the system prompt and combined with Security Brain analysis, RAG context, memory, and tool output to construct the final prompt sent to the local LLM.

4.11 Prompt Builder
Responsibilities:

- Merge Security Brain output
- Merge RAG context
- Merge Memory
- Merge security_consultant.txt
- Merge Tool Results
- Build the final LLM prompt

4.12 Ollama Client
Module: ollama_client.py

Purpose:
Communicate with the locally hosted LLM through Ollama.

Responsibilities:

- Build requests
- Send prompts
- Receive responses
- Handle timeouts
- Detect connection errors
- Return clean model output

Current workflow supports local models such as Qwen and DeepSeek.

4.13 Response Engine

Module: response_engine.py

Purpose:
Prepare the final answer for presentation.

Functions include:

- Clean AI responses
- Remove generic AI disclaimers
- Perform quality checks
- Verify expected report sections
- Produce the final formatted report

Quality checks look for elements such as:

- Security Domain
- Attack Surface
- Attacker Perspective
- Testing Methodology
- Validation
- Evidence
- Risk Assessment
- Remediation

5. Data Flow

Question
↓
Intent
↓
Decision
↓
Knowledge Retrieval
↓
Tool Execution
↓
Prompt Generation
↓
LLM
↓
Quality Validation
↓
Report

6. Folder Structure (Current)

```
CyberAI/
│
├── modules/
│   ├── decision_engine.py
│   ├── intent_router.py
│   ├── security_brain.py
│   ├── tool_advisor.py
│   ├── tool_executor.py
│   ├── tool_output_analyzer.py
│   ├── prompt_builder.py
│   ├── rag_engine.py
│   ├── memory_manager.py
│   ├── response_engine.py
│   └── ollama_client.py
│
├── rag/
├── vectordb/
├── prompts/
│   security_consultant.txt
├── reports/
├── logs/
├── scripts/
│   cyberai_offensive_v7.py (Main Controller)
```

7. Design Principles

CyberAI follows several core principles:

- **Privacy First** – All reasoning and data remain local.
- **Security-Focused** – Designed specifically for cybersecurity workflows.
- **Modular Architecture** – Components can evolve independently.
- **Evidence-Based Reporting** – Encourage reproducible findings.
- **Decision-Driven Execution** – Avoid unnecessary LLM calls.
- **Extensible Knowledge Base** – RAG can grow with new security content.

- **Professional Methodology** – Responses follow offensive security best practices rather than generic AI chat.

8. Current Capabilities

- Local AI
- RAG
- Memory
- Tool Execution
- Tool Analysis
- Security Brain
- Quality Validation
- Report Generation
- Modular Design

9. Current Limitations (v7)

- Limited set of executable tools.
- Single-agent reasoning.
- Sequential workflow.
- Basic memory capabilities.
- No autonomous planning across multiple steps.
- No long-running task orchestration.

These limitations are acceptable for v7 and provide a stable foundation for future versions.

10. Future Vision (v8 and Beyond)

The long-term roadmap includes:

- Multi-agent collaboration (Planner, Researcher, Tool Operator, Reviewer).
- Autonomous penetration-testing workflows.
- Expanded local tool integrations (e.g., Burp, MobSF, Nuclei, Semgrep, Gitleaks).
- Smarter memory with project awareness.
- Confidence scoring and evidence weighting.
- Interactive remediation guidance.
- Exportable reports (Markdown, HTML, PDF).
- Continuous RAG updates from curated cybersecurity sources.
- A portable **CyberAI OS** image for deployment in isolated environments.